



Lista 4 — Publicação, Testes e Tópicos Avançados

Aulas 28 a 36

Desenvolvimento Móvel

Exercícios sobre Firebase, animações, testes, publicação e arquitetura limpa. Cenários de preparação para produção e distribuição de aplicativos.

Exercício 1

Cenário: App de receitas com Firebase (Auth + Firestore) — versão completa com backend.

1. Integre Firebase Authentication: login com email/senha e Google Sign-In.
2. Use Cloud Firestore: coleção "receitas" com documentos {titulo, ingredientes, modoPreparo, tempoPreparo, userId, createdAt}.
3. Implemente regras de segurança no Firestore: apenas usuários autenticados podem ler; apenas o dono pode editar/deletar.
4. Crie um serviço FirebaseService que abstrai todas as chamadas ao Firebase, facilitando testes e manutenção.

Exercício 2

Cenário: Adicionar animações ao app para melhorar a experiência do usuário.

1. Use AnimatedContainer para criar um card que expande ao ser tocado (mostrando mais detalhes com AnimatedCrossFade).
2. Implemente hero animation: ao tocar em uma imagem na lista, ela faz uma transição fluida para a tela de detalhes (Hero widget).
3. Crie uma animação customizada com AnimationController + Tween: um botão pulsando para chamar atenção.
4. Explique o pipeline de renderização do Flutter: build → layout → paint → compositing → rasterize. O que causa repaints desnecessários? Use DevTools > Performance para identificar.

Exercício 3

Cenário: Preparar app para publicação na Google Play Store e App Store.

1. Configure o build: gere keystore (Android), configure build.gradle (versionCode, versionName, minSdk, targetSdk).
2. Crie ícones adaptáveis (Android) e ícones completos (iOS) com diferentes resoluções.
3. Teste em dispositivos reais: pelo menos 3 modelos Android e 2 iPhones. Documente problemas encontrados.
4. Liste os requisitos da Play Store e App Store: política de privacidade, classificação indicativa, capturas de tela, descrição, categorias.

Exercício 4

Cenário: Garantir qualidade com testes automatizados no Flutter.

1. Escreva unit tests para um CounterProvider: verifique increment() e decrement(). Use flutter_test.
2. Crie widget test para um botão CounterScreen: verifique se o texto exibe "0" inicialmente e "1" após clicar.
3. Implemente integration test para o fluxo de login: preenche email/senha, clica em "Entrar", verifica se navegou para Home.
4. Configure o testes no CI (GitHub Actions): workflow que roda flutter analyze e flutter test a cada push.

Requisitos

Flutter SDK. Conta Firebase (gratuita). Contas Google Play (US\$25) e Apple Developer (US\$99/ano) — opcionais para testes.